VNU-HCM

University of Science

faculty of information technology

Data Structure

Project: Data Structure Visualization

Courses: CS163

Students:

Huy, Tran Quoc (25125053)

Thang, Tran Quyet (25125034)

Loi, Tran Thanh (25125059)

Lam, Nguyen Son (25125058)

April 28, 2026

Contents

1 Overview **3**

1.1 Project Description 3

1.2 Project objectives 3

2 Requirments Specification **3**

2.1 Project Models 3

2.1.1 Screen requirments 3

2.1.2 UI components 4

2.2 Background representations 4

2.3 Movements 5

2.3.1 Singly linked list (Insert, delete, search, update) 5

2.3.2 Trie (Insert, Search, Delete) 13

2.3.3 Heap (Insert, Delete Max) 17

2.3.4 Min Spanning Tree (Kruskal, Prim) 21

3 Design **23**

3.1 Architecture Diagram 23

3.2 Design Decisions 24

4 Implement Details **27**

4.1 Code Organization 27

4.1.1 Structs and Classes Logic 28

4.1.2 Set up Visual Studio 29

4.2 Testing and Deployment 30
4.2.1 Testing 30
4.2.2 Deployment 30
4.3 Issues and Limitations 31
4.4 Technical Problems and Solutions 31

5 Work Division **32**

6 Future Improvements **32**

7 Tutorial **32**
7.1 Installation 32
7.2 How to Study 33
7.3 Saving 34

8 Conclusion **34**

9 References **35**

1 Overview

1.1 Project Description

This project is an interactive data structure visualization built with C++ and Raylib graphics library, designed to to help students how core data structures and algorithms work. The application features four visualization modules: Singly Linked List, Trie, Heap, and Graph - Minimum Spanning Tree. It includes a full GUI with a main menu, settings, audio feedback, and screen transitions offering an accessible and engaging learning experience for CS163 students.

1.2 Project objectives

1. Implement core data structure visualizations (Singly Linked List, Trie, MST) with animated, step-by-step operations to illustrate insertion, deletion, search, update, ...
2. Build an intuitive GUI — providing smooth screen navigation, responsive controls, audio feedback, and customizable settings to ensure a user-friendly experience.
3. Including code panel following step - by - step for each alogrithms.

2 Requirments Specification

2.1 Project Models

2.1.1 Screen requirments

The screen includes:

- Main menu consisting of four features.
- Home screen choosing alogrithms.
- Singly Link list screen.
- Trie screen.
- Heap screen.

- Graph screen.
- Settings screen.
- About us screen.

2.1.2 UI components

The UI includes:

- Buttons.
- Card to choose four alogrithms.
- Input field to enter value.
- Camera to pan/ zoom in MST and Trie.
- Font.
- Color to make the perfect background.
- Audio manager to edit music/volume
- App settings to save themes.

2.2 Background representations

The background is represented in `app_settings.h`

```
struct AppSettings {
    float musicVolume  = 0.4f;
    float sfxVolume    = 0.7f;
    int   bgColorIdx   = 0;
    int   musicTrackIdx = 1;   // 0 = no music, 1 = music1, 2 = music2

    static const Color  bgColors[];
    static const char*  bgColorNames[];
    static const int    bgColorCount;
```

```

    Color GetBG() const { return bgColors[bgColorIdx]; }
};
```

Listing 1: The background choice is well define in app_settings.h

2.3 Movements

2.3.1 Singly linked list (Insert, delete, search, update)

Insert step by step animation in `linked_list_screen.cpp`

```
void LinkedListScreen::StartInsertStep(int idx, int v) {
    LLNode nd;
    nd.value = v; nd.alpha = 0.0f; nd.state = LLState::Normal;
    nd.x  = nodes.empty() ? 640.0f : (idx == 0 ? nodes[0].tx : nodes[idx
   > 0 ? idx-1 : 0].tx);
    nd.tx = nd.x;
    nd.y  = NODE_Y - 80.0f; nd.ty = NODE_Y;
    nodes.insert(nodes.begin() + idx, nd);
    LayoutNodes();
    for (auto& n : nodes) n.state = LLState::Normal;

    stepOp    = StepOp::Insert;
    stepActive = true;
    stepPhase  = 0;
    stepTimer  = 0.7f;
    stepIdx    = idx;
    input.Clear();
}
```

Listing 2: The animation of Start Insert step is well-defined in linked_list_screen.cpp

```
if (stepOp == StepOp::Insert) {
        stepPhase++;
```

```
        if (stepPhase == 1 && stepIdx == 0) stepPhase = 2; // no predecessor for head

        for (auto& n : nodes) n.state = LLState::Normal;
        switch (stepPhase) {
            case 1: // predecessor only (orange)
                nodes[stepIdx - 1].state = LLState::Predecessor;
                stepTimer = 0.7f; break;
            case 2: // arrows appear + new node (green); predecessor stays orange
                if (stepIdx > 0) nodes[stepIdx - 1].state = LLState::Predecessor;
                nodes[stepIdx].state = LLState::Highlighted;
                stepTimer = 0.7f; break;
            case 3: // successor appears (blue); others stay
                if (stepIdx > 0) nodes[stepIdx - 1].state = LLState::Predecessor;
                nodes[stepIdx].state = LLState::Highlighted;
                if (stepIdx + 1 < (int)nodes.size())
                    nodes[stepIdx + 1].state = LLState::Successor;
                stepTimer = 0.7f; break;
            default:
                stepActive = false; stepOp = StepOp::None;
                char buf[64];
                snprintf(buf, sizeof(buf), "Inserted %d at index %d.", nodes[stepIdx].value, stepIdx);
                SetMsg(buf, Pal::BtnSuccess, 3.0f);
                break;
        }
        return;
    }
```

Listing 3: Advanced insert step by step

Insert instant in `linked_list_screen.cpp`

```
void LinkedListScreen::InstantInsert(int idx, int v) {
    LLNode nd;
    nd.value = v; nd.alpha = 1.0f; nd.state = LLState::Normal;
    nd.x = nd.tx = nodes.empty() ? 640.0f : (idx == 0 ? nodes[0].tx :
   nodes[idx-1].tx);
    nd.y = nd.ty = NODE_Y;
    nodes.insert(nodes.begin() + idx, nd);
    LayoutNodes();
    for (auto& n : nodes) { n.x = n.tx; n.y = n.ty; }
    char buf[64]; snprintf(buf, sizeof(buf), "Inserted %d at index %d (
   Instant).", v, idx);
    SetMsg(buf, Pal::BtnSuccess, 3.0f);
    input.Clear();
}
```

Listing 4: The animation of Start Insert instant is well-defined in linked_list_screen.cpp

Delete step by step animation

```
void LinkedListScreen::StartDeleteStep(int idx) {
    for (auto& n : nodes) n.state = LLState::Normal;

    stepOp     = StepOp::Delete;
    stepActive = true;
    stepPhase  = 0;
    stepIdx    = idx;

    // Phase 0: if idx==0 mark immediately as Removing, else start
   traversal
    if (idx == 0) {
        nodes[0].state = LLState::Removing;
        stepTimer = 0.7f;
    } else {
        nodes[0].state = LLState::Highlighted;
```

```
        stepTimer = 0.7f;
    }
    input.Clear();
}
```

Listing 5: The animation of Start Delete step by step is well-defined in linked_list_screen.cpp

```
if (stepOp == StepOp::Delete) {
        stepPhase++;
        for (auto& n : nodes) n.state = LLState::Normal;

        if (stepPhase < stepIdx) {
            // Still traversing move pointer to next node
            nodes[stepPhase].state = LLState::Highlighted;
            stepTimer = 0.7f;

        } else if (stepPhase == stepIdx) {
            // Reached target mark for deletion
            nodes[stepIdx].state = LLState::Removing;
            stepTimer = 0.7f;

        } else if (stepPhase == stepIdx + 1) {
            // Erase node, show predecessor (orange) only
            nodes.erase(nodes.begin() + stepIdx);
            LayoutNodes();
            if (stepIdx - 1 >= 0)
                nodes[stepIdx - 1].state = LLState::Predecessor;
            stepTimer = 0.7f;

        } else if (stepPhase == stepIdx + 2) {
            // Keep predecessor (orange), reveal successor (blue)
            if (stepIdx - 1 >= 0)
                nodes[stepIdx - 1].state = LLState::Predecessor;
            if (stepIdx < (int)nodes.size())
```

```
                nodes[stepIdx].state = LLState::Successor;
            stepTimer = 0.7f;

        } else {
            // Done
            for (auto& n : nodes) n.state = LLState::Normal;
            stepActive = false; stepOp = StepOp::None;
            SetMsg("Deleted successfully.", Pal::BtnDanger, 3.0f);
        }
    }
```

Listing 6: Advanced delete step by step

Delete instant in `linked_list_screen.cpp`

```
void LinkedListScreen::InstantDelete(int idx) {
    int val = nodes[idx].value;
    nodes.erase(nodes.begin() + idx);
    LayoutNodes();
    for (auto& n : nodes) { n.x = n.tx; n.y = n.ty; }
    char buf[64]; snprintf(buf, sizeof(buf), "Deleted %d at index %d (
   Instant).", val, idx);
    SetMsg(buf, Pal::BtnDanger, 3.0f);
    input.Clear();
}
```

Listing 7: The animation of delete instant is well-defined in linked_list_screen.cpp

Search step by step animation

```
void LinkedListScreen::StartSearchStep(int v) {
    for (auto& n : nodes) n.state = LLState::Normal;
    stepOp     = StepOp::Search;
    stepActive = true;
    stepPhase  = 0;
    stepVal    = v;
    nodes[0].state = LLState::Highlighted;
```

```
    stepTimer  = 0.7f;
    input.Clear();
}}
```

Listing 8: The animation of Start Search step by step is well-defined in linked_list_screen.cpp

```
if (stepOp == StepOp::Search) {
        // Sentinel found display phase done
        if (stepPhase == (int)nodes.size()) {
            stepActive = false; stepOp = StepOp::None;
            return;
        }
        for (auto& n : nodes) n.state = LLState::Normal;
        if (nodes[stepPhase].value == stepVal) {
            nodes[stepPhase].state = LLState::Found;
            SetMsg("Found!", Pal::BtnSuccess, 3.0f);
            stepPhase = (int)nodes.size(); // sentinel show found code line for one phase
            stepTimer = 0.7f;
            return;
        }
        stepPhase++;
        if (stepPhase >= (int)nodes.size()) {
            stepActive = false; stepOp = StepOp::None;
            SetMsg("Not found!", Pal::BtnDanger, 3.0f);
            return;
        }
        nodes[stepPhase].state = LLState::Highlighted;
        stepTimer = 0.7f;
    }
```

Listing 9: Advanced search step by step

Search instant in `linked_list_screen.cpp`

```
else if (doSearch) {
```

```
        int v; if (parseVal(v)) {
            if (nodes.empty()) { SetMsg("List is empty!", {229,57,53,255}); }
            else if (isStepByStep) { StartSearchStep(v); }
            else {
                bool found = false;
                for (auto& nd : nodes) nd.state = LLState::Normal;
                for (auto& nd : nodes) {
                    if (nd.value == v) { nd.state = LLState::Found; found = true; }
                }
                SetMsg(found ? "Found!" : "Not found!", found ? Pal::BtnSuccess : Pal::BtnDanger);
                input.Clear();
            }
        }
    }
```

Listing 10: The animation of search instant is well-defined in linked_list_screen.cpp

Update step by step animation

```
void LinkedListScreen::StartUpdateStep(int idx, int newVal) {
    for (auto& n : nodes) n.state = LLState::Normal;
    stepOp     = StepOp::Update;
    stepActive = true;
    stepPhase  = 0;
    stepIdx    = idx;
    stepNewVal = newVal;
    nodes[0].state = LLState::Highlighted;
    stepTimer  = 0.7f;
    input.Clear();
}
```

Listing 11: The animation of Start Update step is well-defined in linked_list_screen.cpp

```
if (stepOp == StepOp::Search) {
        // Sentinel found display phase done
        if (stepPhase == (int)nodes.size()) {
            stepActive = false; stepOp = StepOp::None;
            return;
        }
        for (auto& n : nodes) n.state = LLState::Normal;
        if (nodes[stepPhase].value == stepVal) {
            nodes[stepPhase].state = LLState::Found;
            SetMsg("Found!", Pal::BtnSuccess, 3.0f);
            stepPhase = (int)nodes.size(); // sentinel show found code line for one phase
            stepTimer = 0.7f;
            return;
        }
        stepPhase++;
        if (stepPhase >= (int)nodes.size()) {
            stepActive = false; stepOp = StepOp::None;
            SetMsg("Not found!", Pal::BtnDanger, 3.0f);
            return;
        }
        nodes[stepPhase].state = LLState::Highlighted;
        stepTimer = 0.7f;
    }
```

Listing 12: Advanced step by step

Update instant in `linked_list_screen.cpp`

```
else if (doUpdate) {
        if (input.IsEmpty()) { SetMsg("Format: 'index value'  e.g. '2 50'", {229,57,53,255}); }
        else {
            int idx, newVal;
```

```
            if (sscanf(input.text.c_str(), "%d %d", &idx, &newVal) != 2)
                SetMsg("Format: 'index value'  e.g. '2 50'", {229,57,53,255});
            else if (nodes.empty())
                SetMsg("List is empty!", {229,57,53,255});
            else if (idx < 0 || idx >= (int)nodes.size()) {
                char buf[64]; snprintf(buf, sizeof(buf), "Index out of range (0-%d)!", (int)nodes.size()-1);
                SetMsg(buf, {229,57,53,255});
            } else if (isStepByStep) {
                StartUpdateStep(idx, newVal);
            } else {
                for (auto& nd : nodes) nd.state = LLState::Normal;
                nodes[idx].value = newVal;
                nodes[idx].state = LLState::Normal;
                char buf[64]; snprintf(buf, sizeof(buf), "Updated index %d to %d.", idx, newVal);
                SetMsg(buf, Pal::Teal);
                input.Clear();
            }
        }
    }
```

Listing 13: The animation of update instant is well-defined in linked_list_screen.cpp

2.3.2 Trie (Insert, Search, Delete)

Insert step-by-step animation in `trie_screen.cpp`

```
void TrieScreen::StartInsert(const std::string& word) {
    if(!isStepByStep){
        InstantInsert(word);
        return;
    }
```

```
    pendingWord = word;
    currentIdx = 0;
    currentNode = root;
    isAnimating = true;
    isSearching = false;
    stepTimer = 0.0f;
    highlightPath.clear();
    highlightPath.push_back(root);
}
```

Listing 14: The animation of Start Insert step is well-defined in trie_screen.cpp

```
if (isAnimating) {
    if (currentIdx == 0) pool[root].cnt++;
    if (pool[currentNode].children[idx] == -1) {
        int newNodeIdx = (int)pool.size();
        pool.emplace_back(pendingWord[currentIdx]);
        pool[newNodeIdx].x = pool[currentNode].x;
        pool[newNodeIdx].y = pool[currentNode].y;
        pool[newNodeIdx].cnt = 0;
        pool[currentNode].children[idx] = newNodeIdx;
    }
    currentNode = pool[currentNode].children[idx];
    pool[currentNode].cnt++;
    highlightPath.push_back(currentNode);
    currentIdx++;
    Layout();
}
```

Listing 15: Advanced Insert step by step in trie_screen.cpp

Insert instant in `trie_screen.cpp`

```
void TrieScreen::InstantInsert(const std::string& word) {
    int curr = root;
    pool[curr].cnt++;
```

```
    for (char c : word) {
        int idx = (unsigned char)c;
        if (pool[curr].children[idx] == -1) {
            int newNodeIdx = (int)pool.size();
            pool.emplace_back(c);
            pool[newNodeIdx].x = pool[curr].x;
            pool[newNodeIdx].y = pool[curr].y;
            pool[curr].children[idx] = newNodeIdx;
        }
        curr = pool[curr].children[idx];
        pool[curr].cnt++;
    }
    pool[curr].endCount++;
    Layout();
}
```

Listing 16: The animation of Insert instant is well-defined in trie_screen.cpp

Search step-by-step animation in `trie_screen.cpp`

```
void TrieScreen::StartSearch(const std::string& word) {
    pendingWord = word;
    currentIdx = 0;
    currentNode = root;
    isSearching = true;
    isAnimating = false;
    stepTimer = 0.0f;
    highlightPath.clear();
    highlightPath.push_back(root);
}
```

Listing 17: The animation of Start Search step is well-defined in trie_screen.cpp

```
else if (isSearching) {
    if (IsValidChild(currentNode, idx)) {
        currentNode = pool[currentNode].children[idx];
```

```
        highlightPath.push_back(currentNode);
        currentIdx++;
    } else {
        isSearching = false;
        SetMsg(isDeletingStep ? "Word not found to delete"
                              : "Not Found", Pal::BtnDanger);
        isDeletingStep = false;
        highlightPath.clear();
    }
}
```

Listing 18: Advanced Search step by step in trie_screen.cpp

Search instant in `trie_screen.cpp`

```
bool TrieScreen::InstantSearch(const std::string& word) {
    int curr = root;
    highlightPath.clear();
    highlightPath.push_back(root);
    for (char c : word) {
        int idx = (unsigned char)c;
        if (!IsValidChild(curr, idx)) {
            SetMsg("Not Found", Pal::BtnDanger);
            return false;
        }
        curr = pool[curr].children[idx];
        highlightPath.push_back(curr);
    }
    if (pool[curr].endCount > 0) {
        SetMsg("Found!", Pal::BtnSuccess);
        return true;
    }
    SetMsg("Prefix exists, but word not found", Pal::BtnOrange);
    return false;
}
```

Listing 19: The animation of Search instant is well-defined in trie_screen.cpp

Delete animation in `trie_screen.cpp`

```cpp
void TrieScreen::Delete(const std::string& word) {
    if (!InstantSearch(word)) return;
    int cur = root;
    pool[cur].cnt--;
    for (char c : word) {
        int idx = (unsigned char)c;
        int next = pool[cur].children[idx];
        pool[next].cnt--;
        if (pool[next].cnt <= 0) {
            pool[cur].children[idx] = -1;
        }
        cur = next;
    }
    if (cur != -1 && pool[cur].endCount > 0) {
        pool[cur].endCount--;
    }
    highlightPath.clear();
    Layout();
    SetMsg("Word instance deleted!", Pal::BtnSuccess);
}
```

Listing 20: The animation of Delete is well-defined in trie_screen.cpp

2.3.3 Heap (Insert, Delete Max)

Insert step-by-step animation in `heap_screen.cpp`

```cpp
if (doingInsert) {
    if (idCurrent == 0) {
        doingInsert = false;
        animA = animB = animC = -1;
```

```
        ComputePositions();
        snprintf(buf, sizeof(buf), "Inserted %d.", heap[idCurrent]);
        SetMsg(buf);
    } else {
        int parent = (idCurrent - 1) >> 1;
        animA = idCurrent;
        animB = parent;
        if (!isHighlight) {
            ComputePositions();
            isHighlight = true;
        } else {
            if (heap[idCurrent] > heap[parent]) {
                std::swap(heap[idCurrent], heap[parent]);
                std::swap(vis[idCurrent], vis[parent]);
                idCurrent = parent;
                animA = animB = animC = -1;
                ComputePositions();
                isHighlight = false;
            } else {
                doingInsert = false;
                snprintf(buf, sizeof(buf), "Inserted %d.", heap[idCurrent
    ]);
                idCurrent = 0;
                SetMsg(buf);
            }
        }
    }
}
```

Listing 21: The animation of Start Insert step is well-defined in heap_screen.cpp

Insert instant in `heap_screen.cpp`

```
void HeapScreen::InstantInsert() {
    int n = (int)heap.size() - 1;
```

```
    while (n && heap[n] > heap[(n - 1) / 2]) {
        std::swap(heap[n], heap[(n - 1) / 2]);
        n = (n - 1) / 2;
    }
    ComputePositions();
    char buf[32];
    snprintf(buf, sizeof(buf), "Inserted %d.", heap[n]);
    SetMsg(buf);
}
```

Listing 22: The animation of Insert instant is well-defined in heap_screen.cpp

Delete Max step-by-step animation in `heap_screen.cpp`

```
if (doingDelete) {
    int n = (int)heap.size();
    int largest = idCurrent;
    int l = 2 * idCurrent + 1;
    int r = 2 * idCurrent + 2;
    animA = idCurrent; animB = l; animC = r;
    if (!isHighlight) {
        ComputePositions();
        isHighlight = true;
    } else {
        if (l < n && heap[l] > heap[largest]) largest = l;
        if (r < n && heap[r] > heap[largest]) largest = r;
        if (largest == idCurrent) {
            animA = animB = animC = -1;
            idCurrent = 0;
            doingDelete = false;
            ComputePositions();
            SetMsg("Deleted max.", Pal::BtnDanger);
        } else {
            std::swap(heap[idCurrent], heap[largest]);
            std::swap(vis[idCurrent], vis[largest]);
```

```
            idCurrent = largest;
            ComputePositions();
            isHighlight = false;
        }
    }
}
```

Listing 23: The animation of Delete Max step is well-defined in heap_screen.cpp

Delete Max instant in `heap_screen.cpp`

```
void HeapScreen::InstantDel() {
    heap[0] = heap.back();
    heap.pop_back();
    int n = (int)heap.size();
    int id = 0;
    while (true) {
        int largest = id;
        int l = 2 * id + 1;
        int r = 2 * id + 2;
        if (l < n && heap[l] > heap[largest]) largest = l;
        if (r < n && heap[r] > heap[largest]) largest = r;
        if (largest == id) {
            ComputePositions();
            SetMsg("Deleted max (Instant).", Pal::BtnDanger);
            break;
        } else {
            std::swap(heap[id], heap[largest]);
            id = largest;
        }
    }
}
```

Listing 24: The animation of Delete Max instant is well-defined in heap_screen.cpp

2.3.4 Min Spanning Tree (Kruskal, Prim)

Kruskal step-by-step animation in `graph_screen.cpp`

```
void GraphScreen::RunKruskal() {
    mstVisitOrder.clear();
    std::vector<int> order;
    for (int i = 0; i < (int)edges.size(); i++)
        if (edges[i].visible) order.push_back(i);
    std::sort(order.begin(), order.end(),
        [&](int a, int b){ return edges[a].w < edges[b].w; });

    std::vector<int> parent(nodeCount);
    std::iota(parent.begin(), parent.end(), 0);
    std::function<int(int)> find = [&](int x) -> int {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    };

    mstSteps.clear();
    int cumWeight = 0;
    mstSteps.push_back({-1, false, cumWeight, 0, false});
    for (int idx : order) {
        const GEdge& e = edges[idx];
        if (!nodes[e.u].visible || !nodes[e.v].visible) continue;
        mstSteps.push_back({idx, false, cumWeight, 1, false});
        int pu = find(e.u), pv = find(e.v);
        mstSteps.push_back({idx, false, cumWeight, 2, false});
        bool added = (pu != pv);
        if (added) {
            parent[pu] = pv;
            cumWeight += e.w;
            mstSteps.push_back({idx, true,  cumWeight, 3, true});
        } else {
            mstSteps.push_back({idx, false, cumWeight, 4, true});
```

```
        }
    }
    mstSteps.push_back({-1, false, cumWeight, 5, false});
    mstCurrentStep = 0; mstActive = true;
    mstStepTimer = kMstStepDuration;
    mstAlgoType = MSTAlgorithmType::Kruskal;
    SetMsg("Kruskal: running step by step...", Pal::TxtMid, 3.0f);
}
```

Listing 25: The animation of Kruskal is well-defined in graph_screen.cpp

Prim step-by-step animation in `graph_screen.cpp`

```
void GraphScreen::RunPrim() {
    mstVisitOrder.clear();
    int start = -1, visibleCount = 0;
    for (int i = 0; i < nodeCount; i++)
        if (nodes[i].visible) { if (start == -1) start = i; visibleCount
   ++; }
    if (start == -1) { SetMsg("No visible nodes.", {200,60,60,255}, 3.0f)
   ; return; }

    mstAlgoType = MSTAlgorithmType::Prim;
    std::vector<bool> inTree(nodeCount, false);
    inTree[start] = true;
    mstVisitOrder.push_back(start);
    mstSteps.clear();

    int cumWeight = 0, added = 0;
    mstSteps.push_back({-1, false, cumWeight, 0, false});
    while (added < visibleCount - 1) {
        mstSteps.push_back({-1, false, cumWeight, 1, false});
        int bestIdx = -1, bestW = INT_MAX;
        for (int i = 0; i < (int)edges.size(); i++) {
            const GEdge& e = edges[i];
```

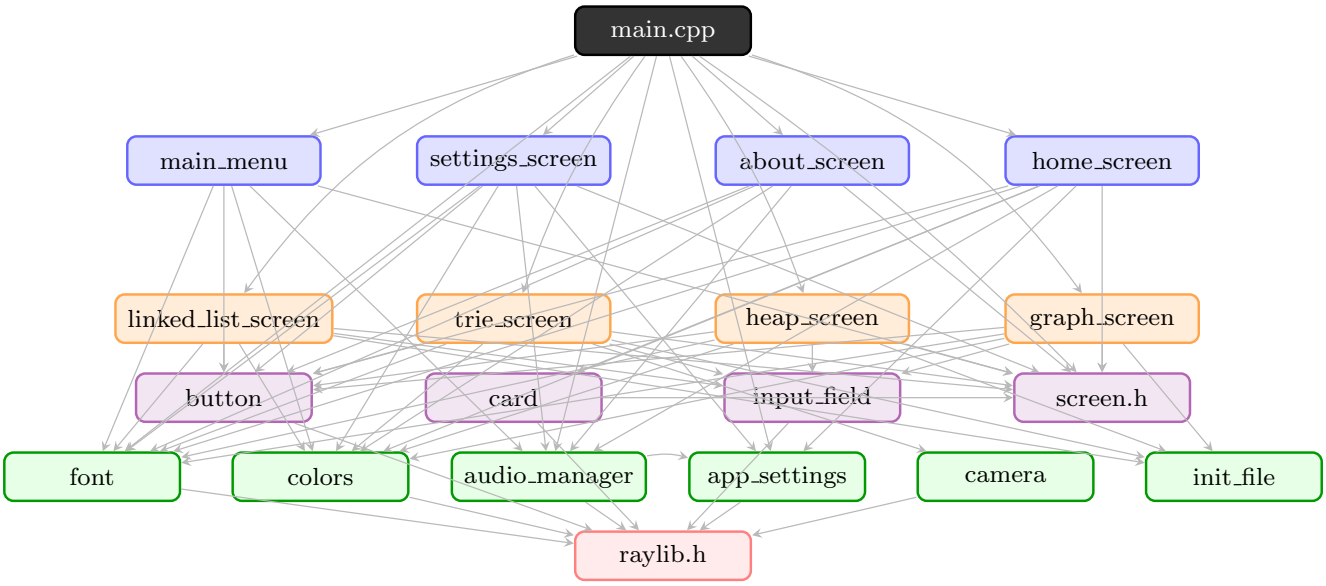
```
            if (!e.visible || !nodes[e.u].visible || !nodes[e.v].visible
                || inTree[e.u] == inTree[e.v]) continue;
            if (e.w < bestW) { bestW = e.w; bestIdx = i; }
        }
        if (bestIdx >= 0)
            mstSteps.push_back({bestIdx, false, cumWeight, 2, false});
        if (bestIdx == -1) break;
        int u = edges[bestIdx].u, v = edges[bestIdx].v;
        int nextNode = inTree[u] ? v : u;
        inTree[u] = inTree[v] = true;
        if (nextNode >= 0 && nextNode < nodeCount)
            mstVisitOrder.push_back(nextNode);
        cumWeight += edges[bestIdx].w;
        mstSteps.push_back({bestIdx, true, cumWeight, 3, true});
        added++;
    }
    mstSteps.push_back({-1, false, cumWeight, 4, false});
    mstCurrentStep = 0; mstActive = true;
    mstStepTimer = kMstStepDuration;
    SetMsg("Prim: running step by step...", Pal::TxtMid, 3.0f);
}
```

Listing 26: The animation of Prim is well-defined in graph_screen.cpp

3 Design

3.1 Architecture Diagram

This project follows a modular architecture:



3.2 Design Decisions

Linked List uses `std::vector<LLNode>` instead of a pointer-based list, providing $O(1)$ index access to highlight any node at each animation step without traversal. *Alternative:* classic `Node* next` pointer chain — rejected due to $O(n)$ traversal cost for random node access during animation.

Trie stores all nodes in a flat `std::vector<TrieNode>` pool with `int children[128]` index arrays, avoiding pointer invalidation on reallocation and simplifying the animation loop. *Alternative:* `TrieNode* children[128]` heap-allocated tree — rejected due to pointer invalidation when the container grows.

Heap uses the standard array representation where parent/child relationships follow index arithmetic ($parent = (i-1)/2$). A parallel `HeapVis` vector stores visual state separately from algorithm logic. *Alternative:* explicit binary tree with `left/right` pointers — rejected as unnecessary complexity given the natural array mapping.

Graph stores nodes in a fixed `GNode nodes[26]` array (A–Z) and edges in a `std::vector<GEdge>`. A `visible` flag on both enables soft deletion without reallocation, supporting all three input modes: edge list, adjacency matrix, and adjacency list. *Alternative:* `std::unordered_map` for dynamic node lookup — rejected since a fixed A–Z label set makes a 26-slot array sufficient and faster.

MST pre-computes all steps into a `std::vector<MSTStep>` before animation begins, fully separating algorithm logic from visual timing. *Alternative:* interleaving algorithm execution with the

render loop — rejected due to risk of algorithm and visual state going out of sync.

UI Components (`Button`, `Card`, `InputField`) are implemented as reusable structs with `Update()` and `Draw()`, keeping UI logic self-contained across all screens. *Alternative:* an inheritance-based widget hierarchy — rejected as over-engineered for a fixed set of simple components.

GUI Framework uses **Raylib** as the sole rendering backend, drawing all interface elements (buttons, panels, text, shapes) directly via immediate-mode 2D draw calls each frame. *Alternative:* a retained-mode GUI library such as Dear ImGui or Qt — rejected because Raylib provides full control over layout, animation, and visual style with minimal overhead, which is essential for smooth step-by-step algorithm visualization at 120 FPS.

4 Implement Details

4.1 Code Organization

File	Description
`main.cpp`	Entry point. Initializes window, audio, and fonts. Runs the main game loop and dispatches `Update()`/`Draw()` to the active screen.
`screen.h`	Defines the `Screen` enum used to identify and transition between all screens.
`main_menu.cpp/h`	Main menu screen with navigation to Play, Settings, and About.
`home_screen.cpp/h`	Home screen displaying four module cards for selecting a data structure.
`settings_screen.cpp/h`	Settings screen for adjusting background color, music track, and volume.
`about_screen.cpp/h`	About screen displaying project and team information.
`linked_list_screen.cpp/h`	Visualizes singly linked list with insert, delete, search, and update in step-by-step and instant modes.
`trie_screen.cpp/h`	Visualizes a Trie with insert, search, and delete. Supports camera pan/zoom and file loading.
`heap_screen.cpp/h`	Visualizes a max-heap with insert and delete-max including heapify animation.
`graph_screen.cpp/h`	Visualizes a weighted graph with Kruskal and Prim, supporting edge list, adjacency matrix, and adjacency list input.
`button.cpp/h`	Reusable button with hover, click detection, and customizable colors.
`card.cpp/h`	Clickable card used in the home screen to select a visualizer module.
`input_field.cpp/h`	Text input field with keyboard entry, focus management, and character limit.
`camera.cpp/h`	Pan and zoom controls via mouse drag and scroll wheel for Trie and Graph screens.
`font.cpp/h`	Loads Roboto font variants and provides centered/bounded text drawing helpers.
`colors.h`	Centralized color palette (`Pal::`) shared across all screens.
`app_settings.cpp/h`	Stores user preferences: background color, music track, and volume.
`audio_manager.cpp/h`	Manages music playback and UI sound effects. Reads volume from `AppSettings`.
`init_file.cpp/h`	Loads integer or word lists from `data.txt` for bulk insertion.

Table 1: Description of each source file in the project.

4.1.1 Structs and Classes Logic

LLNode — stores a linked list node's value, visual position `(x, y, tx, ty)`, opacity `alpha`, and highlight state `LLState` (Normal, Highlighted, Found, Removing, Predecessor, Successor).

LinkedListScreen — holds `vector<LLNode> nodes` as core data. Key methods: `StartInsertStep()`, `StartDeleteStep()`, `StartSearchStep()`, `StartUpdateStep()`, `AdvanceStep()`, `InstantInsert()`, `InstantDelete()`, `LayoutNodes()`, `Update()`, `Draw()`.

TrieNode — stores `children[128]` as index array, `cnt`, `endCount`, `leafCount`, and visual fields `(x, y, targetX, targetY, alpha, ch)`.

TrieScreen — manages a flat `vector<TrieNode> pool` and `highlightPath`. Key methods: `StartInsert()`, `StartSearch()`, `Delete()`, `InstantInsert()`, `InstantSearch()`, `Layout()`, `LayoutSubtree()`, `DrawAllNodes()`, `DrawAllEdges()`, `Update()`, `Draw()`.

HeapNodeVis — stores per-node visual state: `x, y, tx, ty, alpha, highlighted`.

HeapScreen — holds `vector<int> heap` and parallel `vector<HeapNodeVis> vis`. Key methods: `InstantInsert()`, `InstantDel()`, `ComputePositions()`, `GetNodePos()`, `GetPseudoCodeLine()`, `Update()`, `Draw()`.

GNode — a graph node with `x, y, label, visible`.

GEdge — a weighted edge with `u, v, w, visible`.

MSTStep — records one algorithm step: `edgeIdx`, `added`, `cumulativeWeight`, `pseudoLine`, `marksDecision`.

GraphScreen — holds `GNode nodes[26]`, `vector<GEdge> edges`, and `vector<MSTStep> mstSteps`. Key methods: `RunKruskal()`, `RunPrim()`, `ApplyInputToGraph()`, `SyncFieldsFromGraph()`, `UpdateMSTAnima`
`HandleGraphInteraction()`, `Update()`, `Draw()`.

Button — `rect, label, baseColor, hoverColor`. Methods: `Update()` returns `true` on click, `Draw()`.

Card — `rect, target (Screen), title, subtitle, color`. Methods: `Update()`, `Draw()`, `DrawIllustrat`

InputField — `rect, text, focused, maxLen, cursorBlink`. Methods: `Update()`, `Draw()`, `Clear()`, `IsEmpty()`.

AppSettings — `musicVolume, sfxVolume, bgColorIdx, musicTrackIdx`. Method: `GetBG()` returns the active background color.

InitFile — static utility. Methods: `loadNumbers()`, `loadWords()` for bulk-loading from `data.txt`.

4.1.2 Set up Visual Studio

This project uses CMake for build configuration. Key Key setup from `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.25)
project("MyGame")

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

file(GLOB_RECURSE SOURCE_FILES ${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp)

add_executable(${PROJECT_NAME} ${SOURCE_FILES})

add_subdirectory("raylib")

add_definitions(-DPROJECT_ROOT_PATH="${CMAKE_CURRENT_SOURCE_DIR}/")

target_link_libraries(${PROJECT_NAME} PUBLIC raylib)
target_include_directories(${PROJECT_NAME} PRIVATE include)

if(APPLE)
    target_link_libraries(${PROJECT_NAME} PUBLIC "-framework CoreFoundation")
endif()

# Copy fonts next to the executable so relative paths work at runtime
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
        "${CMAKE_CURRENT_SOURCE_DIR}/fonts"
        "$<TARGET_FILE_DIR:${PROJECT_NAME}>/fonts"
)

# Copy assets (music, images, etc.) next to the executable
```

```
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
        "${CMAKE_CURRENT_SOURCE_DIR}/assets"
        "$<TARGET_FILE_DIR:${PROJECT_NAME}>/assets"
)
```

Listing 27: CMake Build Configuration

4.2 Testing and Deployment

4.2.1 Testing

Testing was performed manually by running the application and verifying each feature against expected behavior.

- **Linked List** — tested insert, delete, search, and update at head, tail, and arbitrary index in both Step-by-step and Instant modes.
- **Trie** — tested insert, search (exact word, prefix-only, not found), delete, and file loading with camera pan/zoom.
- **Heap** — tested insert and delete-max with heapify verification, overflow handling, and array view consistency.
- **Graph** — tested all three input modes, Kruskal and Prim correctness, node/edge deletion, and drag interaction.
- **GUI** - test well setting up full buttons, musics, fonts,...

4.2.2 Deployment

Requirements: CMake $\geq$ 3.25, a C++17 compiler. Raylib is bundled in `raylib/` — no system installation required.

Build:

```
mkdir build && cd build
cmake ..
```

```
cmake --build .
```

Listing 28: Build commands

The executable is generated in `build/`. CMake automatically copies `fonts/` and `assets/` next to it as a post-build step. Supported platforms: Windows, macOS, and Linux.

4.3 Issues and Limitations

- **Linked List** — The current implementation stores nodes in a `std::vector` without enforcing a maximum size. Inserting too many nodes causes them to overflow the visible screen width, making rightmost nodes unreachable without a proper scroll mechanism.

- **Trie** — The node pool uses `children[128]`, restricting input to 7-bit ASCII characters. Unicode or accented characters beyond index 127 are not handled. The layout algorithm distributes nodes by leaf count, which can produce very wide or very deep trees for skewed inputs, pushing nodes outside the visible area. Case-insensitive search and wildcard matching are not supported.

- **Heap** — Hard-capped at 15 nodes due to screen space constraints — a four-level binary tree already spans the full window width. Only a max-heap is implemented; min-heap and priority queue variants are not available. The delete operation is limited to removing the root (maximum element); deleting an arbitrary node by value is not supported.

- **Graph** — Limited to 26 nodes (A–Z) and 50 edges, which is insufficient for larger real-world graphs. Only undirected graphs with non-negative integer weights are supported; directed graphs and negative weights are not handled. On the algorithm side, only Kruskal and Prim are implemented.

4.4 Technical Problems and Solutions

We have difficulties in using one background for all alogrithms, and prepare it similarly to make it more beautiful. It is difficult to calcute the position of each button so we decided to allow 2 members to do background to make it more beautiful, not complicated prepared in all alogrithms screens.

5 Work Division

Work of each member

Video of project

6 Future Improvements

- **Linked List** — Add scroll support for large lists.
- **Trie** — Extend to full Unicode input. Add case-insensitive search and prefix autocomplete visualization.
- **Heap** — Add zoom/pan support to remove the 15-node cap. Support min-heap and arbitrary node deletion.
- **Graph** — Support directed graphs and negative weights.
- **GUI** — Make the layout fully responsive. Add transition animations, theme toggle, and button tooltips.
- **General** — Persist settings between sessions. Add animation speed control and undo/redo support.

7 Tutorial

7.1 Installation

1. Clone the repository:

```
git clone https://github.com/tranquyetthang27/CS163-Visualize.git
cd CS163-Visualize
```

Listing 29: Clone the repository

2. Build the project:

```
mkdir build && cd build
cmake ..
cmake --build .
```

Listing 30: Build commands

3. Run the executable generated in `build/`. No additional installation is required — `fonts/` and `assets/` are copied automatically by CMake.

7.2 How to Study

Navigation:

- From the **Main Menu**, click *Start* to enter the Home screen, or navigate to *Settings* and *About*.
- On the **Home screen**, click any of the four cards (Linked List, Trie, Heap, Graph) to open that visualizer.
- Press `ESC` or click < *Back* to return to the previous screen at any time.

Using the visualizers:

- Type a value into the input field and click the corresponding operation button (Insert, Delete, Search, Update).
- Toggle between **Step-by-step** and **Instant** mode using the Mode button — Step mode animates each phase one at a time, Instant mode applies the operation immediately.
- Click **Show Code** to display the pseudocode panel, which highlights the active line during each animation step.
- Click **Load File** to bulk-insert values from `data.txt` located at the project root.

Trie and Graph specific:

- Use the **scroll wheel** to zoom and **click and drag** on the background to pan the camera.

- On the Graph screen, drag nodes to reposition them. Click a node or edge to select and edit or delete it.
- Click **Kruskal** or **Prim** to run the MST algorithm step by step on the current graph.

Settings:

- Adjust background color, music track, and volume from the **Settings** screen accessible via the Main Menu.

7.3 Saving

When backing Home then start agian, we set "current = Screen::LinkedList — switch" to "draw llScreen" having already memory, no reset, so data is remain unchanged.

8 Conclusion

This project delivers an interactive data structure visualization built with **C++17** and **Raylib**, applying **Object-Oriented Programming** principles throughout — each screen and UI component is encapsulated as an independent class with clear `Update()` and `Draw()` interfaces, promoting modularity and reusability.

Strengths: clean screen-based architecture, synchronized pseudocode panel, dual step-by-step and instant animation modes, consistent UI component system (`Button`, `Card`, `InputField`), and cross-platform build via CMake.

Limitations: fixed node caps, no Unicode support in Trie, no directed graph or shortest path algorithms, and settings are not persisted between sessions.

Libraries used: Raylib handled all rendering, audio, and input with minimal overhead, making it well-suited for a real-time visualization tool at 120 FPS. CMake ensured reproducible builds across Windows, macOS, and Linux without any system-level dependencies.

Overall, the project reinforced both theoretical understanding of data structures and practical experience in real-time graphics programming and software design.

9 References

References

[1] Raylib - A simple and easy-to-use library to enjoy videogames programming. *https://www.raylib.com/*

[2] NUS data structure visualization. *Data Structure Visualization*. https://visualgo.net/en

[3] CMake Documentation. *https://cmake.org/documentation/*

[4] Background Music *https://freetouse.com/music/category/relaxing*

[5] Sound Effects *https://www.epidemicsound.com/sound-effects/categories/cartoon/*

[6] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms* (3rd ed.). MIT Press.